MODULE 03

Generics —

Write Once, Use for Any Type

Generics look scary at first — the angle brackets <T> throw everyone off. But once you understand the idea, you'll see them everywhere and wonder how you coded without them.

~55 min read | Builds on Module 2 | Most powerful TS feature

1. The Problem Generics Solve

Imagine you need a function that returns the first item of an array. In JavaScript, easy:

JavaScript — no types

```
// JS — works for any array
function first(arr) {
  return arr[0];
}
```

Now in TypeScript, you have a problem. What type do you give arr and the return value?

Two bad approaches

```
// Attempt 1: Use a specific type — but now it ONLY works for strings
function first(arr: string[]): string {
  return arr[0];
}
first([1, 2, 3]); // Error: number[] not assignable to string[]

// Attempt 2: Use any — works, but loses ALL type safety
function first(arr: any[]): any {
  return arr[0];
}
const result = first([1, 2, 3]);
result.toUpperCase(); // No error from TS... but crashes at runtime!
```

Neither approach is good. This is exactly the problem generics solve — let the caller decide the type, while keeping full type safety.

2. Your First Generic — The <T> Syntax

A generic is a type placeholder. You write <T> (or any letter) as a stand-in, and TypeScript fills it in automatically based on what you actually pass:

Generic function — T is inferred automatically

```
//            T is a type placeholder — filled in at call time
//            |
function first<T>(arr: T[]): T {
  return arr[0];
}

// TS infers T = number automatically
const num = first([1, 2, 3]);
//    num is type: number   TS knows this!

// TS infers T = string automatically
const str = first(['a', 'b', 'c']);
//    str is type: string

// TS infers T = boolean automatically
const bool = first([true, false]);
//    bool is type: boolean

// Type safety is preserved!
num.toUpperCase();  // Error: toUpperCase() doesn't exist on number
str.toUpperCase();  // OK — TS knows str is a string
```

> **Why T? What does it stand for?**
>
> T stands for Type — but it's just a convention, not a keyword. You could write <Banana> and it would work the same way. Common conventions: T = Type, K = Key, V = Value, E = Element, R = Return. Use T for most cases.

3. Multiple Type Parameters

Generics can have more than one type placeholder. A classic example is a function that pairs two values together:

Multiple type parameters

```
// Two type parameters: T and U
function pair<T, U>(first: T, second: U): [T, U] {
  return [first, second];
}

const result1 = pair('Alice', 30);
// result1 is: [string, number]

const result2 = pair(true, ['a', 'b']);
// result2 is: [boolean, string[]]

// Real-world: a function that maps one type to another
function mapValue<Input, Output>(
  value: Input,
  transform: (val: Input) => Output
): Output {
  return transform(value);
}

const length = mapValue('hello', s => s.length);
// Input = string, Output = number
// length is type: number
```

4. Generic Interfaces — Reusable Shapes

Generics aren't limited to functions. You can make interfaces generic too. This is extremely common for API responses, state containers, and data wrappers:

Generic interface — ApiResponse<T>

```
// A generic API response wrapper — works for any data type
interface ApiResponse<T> {
  data:    T;         // whatever T is
  status:  number;
  message: string;
  success: boolean;
}
```

```
// Define your data types
interface User  { id: number; name: string; }
interface Product { id: number; price: number; }

// Use ApiResponse<T> with concrete types
const userResponse:    ApiResponse<User>    = {
  data:    { id: 1, name: 'Alice' },
  status:  200,
  message: 'OK',
  success: true,
};

const productResponse: ApiResponse<Product> = {
  data:    { id: 42, price: 999 },
  status:  200,
  message: 'OK',
  success: true,
};

// TS knows the exact shape of .data in each case
userResponse.data.name;    // OK — string
productResponse.data.price; // OK — number
userResponse.data.price;   // Error! User doesn't have 'price'
```

> **You have already used generic interfaces!**
>
> Array<T> is a built-in generic interface. When you write string[], that's shorthand for Array<string>. Promise<T> is another one — Promise<User> means it resolves to a User. Generics are everywhere in TypeScript.

5. Generic Constraints — Narrowing What T Can Be

Sometimes you want T to be flexible, but not infinitely so. You can constrain T to only allow types that have certain properties using the extends keyword:

Generic constraints with extends

```
// This function wants to print any object's 'name' property
// But T is totally unconstrained — what if T is a number?
function printName<T>(item: T): void {
  console.log(item.name); // Error: 'name' doesn't exist on T
}
```

```
// Solution: constrain T to types that HAVE a 'name' property
function printName<T extends { name: string }>(item: T): void {
  console.log(item.name); // OK — TS knows T has a name string
}

printName({ name: 'Alice', age: 30 });    // OK
printName({ name: 'Laptop', price: 999 }); // OK
printName({ age: 30 });                    // Error: no 'name' prop
printName(42);                             // Error: number has no 'name'
```

Constraining to an Interface

Constraining T to an interface

```
interface HasId {
  id: number;
}

// T must be something that has an id
function findById<T extends HasId>(items: T[], id: number): T | undefined
{
  return items.find(item => item.id === id);
}

interface User    { id: number; name: string; }
interface Product { id: number; price: number; }

const users:    User[]    = [{ id: 1, name: 'Alice' }];
const products: Product[] = [{ id: 99, price: 49 }];

findById(users,    1);  // returns User | undefined
findById(products, 99); // returns Product | undefined
// One function, works for any type with an id!
```

6. Default Type Parameters

Just like function parameters can have default values, type parameters can too. This lets you make a generic interface that works without specifying T every time:

Default type parameters

```
// T defaults to string if not specified
interface Box<T = string> {
```

```
  value: T;
  label: string;
}

const nameBox: Box          = { value: 'Alice', label: 'Name' };
//             Box = Box<string> (default applied)

const ageBox:  Box<number>  = { value: 30, label: 'Age' };
//             Box<number>  (explicit override)

const flagBox: Box<boolean> = { value: true, label: 'Active' };
```

7. Built-in Generic Types You'll Use Daily

TypeScript ships with many built-in generics. These are the ones you'll encounter constantly in React development:

Array<T> and Promise<T>

Array<T> and Promise<T>

```
// Array<T> — you've seen this as T[]
const names: Array<string>  = ['Alice', 'Bob'];   // same as string[]
const ids:   Array<number>  = [1, 2, 3];          // same as number[]

// Promise<T> — what an async function returns
async function fetchUser(id: number): Promise<User> {
  const res  = await fetch(`/api/users/${id}`);
  const user = await res.json();
  return user;
}

const user = await fetchUser(1);
// user is type: User  — TS knows exactly what came back!
```

Record<K, V> — Typed Dictionaries

Record<K, V>

```
// Record<Key, Value> — for objects used as dictionaries/maps
const scores: Record<string, number> = {
  alice: 95,
  bob:   87,
```

```
  carol: 92,
};

// With literal union keys — very strict!
type Role = 'admin' | 'user' | 'guest';
const permissions: Record<Role, string[]> = {
  admin: ['read', 'write', 'delete'],
  user:  ['read', 'write'],
  guest: ['read'],
  // TS forces you to handle ALL roles — no missing keys!
};
```

Partial<T> and Required<T>

Partial<T> and Required<T>

```
interface User {
  id:    number;
  name:  string;
  email: string;
}

// Partial<T> — makes ALL properties optional
// Perfect for update/patch operations where you send only changed fields
function updateUser(id: number, changes: Partial<User>): void {
  // changes can have any combination of User's fields
}
updateUser(1, { name: 'Bob' });              // OK — only name
updateUser(1, { email: 'b@b.com' });         // OK — only email
updateUser(1, { name: 'Bob', email: 'b@b.com' }); // OK — both

// Required<T> — opposite: makes ALL properties required
interface Config {
  debug?:   boolean;
  timeout?: number;
}
type StrictConfig = Required<Config>;
// { debug: boolean; timeout: number } — no more optionals
```

8. Real-World Example — A Typed useFetch Hook

Here's a realistic generic pattern you'll write in React TypeScript — a data-fetching utility that works for any API endpoint:

Generic fetch wrapper — used in every React TS app

```
// A generic fetch wrapper — works for ANY endpoint

interface FetchState<T> {
  data:    T | null;
  loading: boolean;
  error:   string | null;
}

// Generic async fetcher
async function fetchData<T>(url: string): Promise<FetchState<T>> {
  try {
    const res  = await fetch(url);
    const data = await res.json() as T;
    return { data, loading: false, error: null };
  } catch (err) {
    return { data: null, loading: false, error: String(err) };
  }
}

// Usage — fully type-safe for different endpoints
interface User    { id: number; name: string; }
interface Product { id: number; price: number; name: string; }

const userState    = await fetchData<User>('/api/user/1');
// userState.data is: User | null
if (userState.data) {
  console.log(userState.data.name);  // TS knows: string
}

const productState = await fetchData<Product>('/api/products/99');
// productState.data is: Product | null
if (productState.data) {
  console.log(productState.data.price); // TS knows: number
}
```

Module Summary

Everything you learned in this module:

- Generics solve the 'I want type safety but also flexibility' problem

- Syntax: `function f<T>(arg: T): T` — T is a placeholder filled at call time
- TypeScript infers T automatically — you rarely need to specify it explicitly
- Multiple type params: `<T, U>` — each can be different types
- Generic interfaces: `interface ApiResponse<T> { data: T }`
- Constraints: `<T extends { name: string }>` — T must have those properties
- Default type params: `<T = string>` — T defaults to string if not given
- Built-in generics you'll use: `Array<T>`, `Promise<T>`, `Record<K,V>`, `Partial<T>`, `Required<T>`

> Next: Module 4

Enums, Narrowing & Type Guards

Enums for named constants, type narrowing to handle unions safely, typeof / instanceof guards, and discriminated unions — the pattern that makes complex state management in React bulletproof.